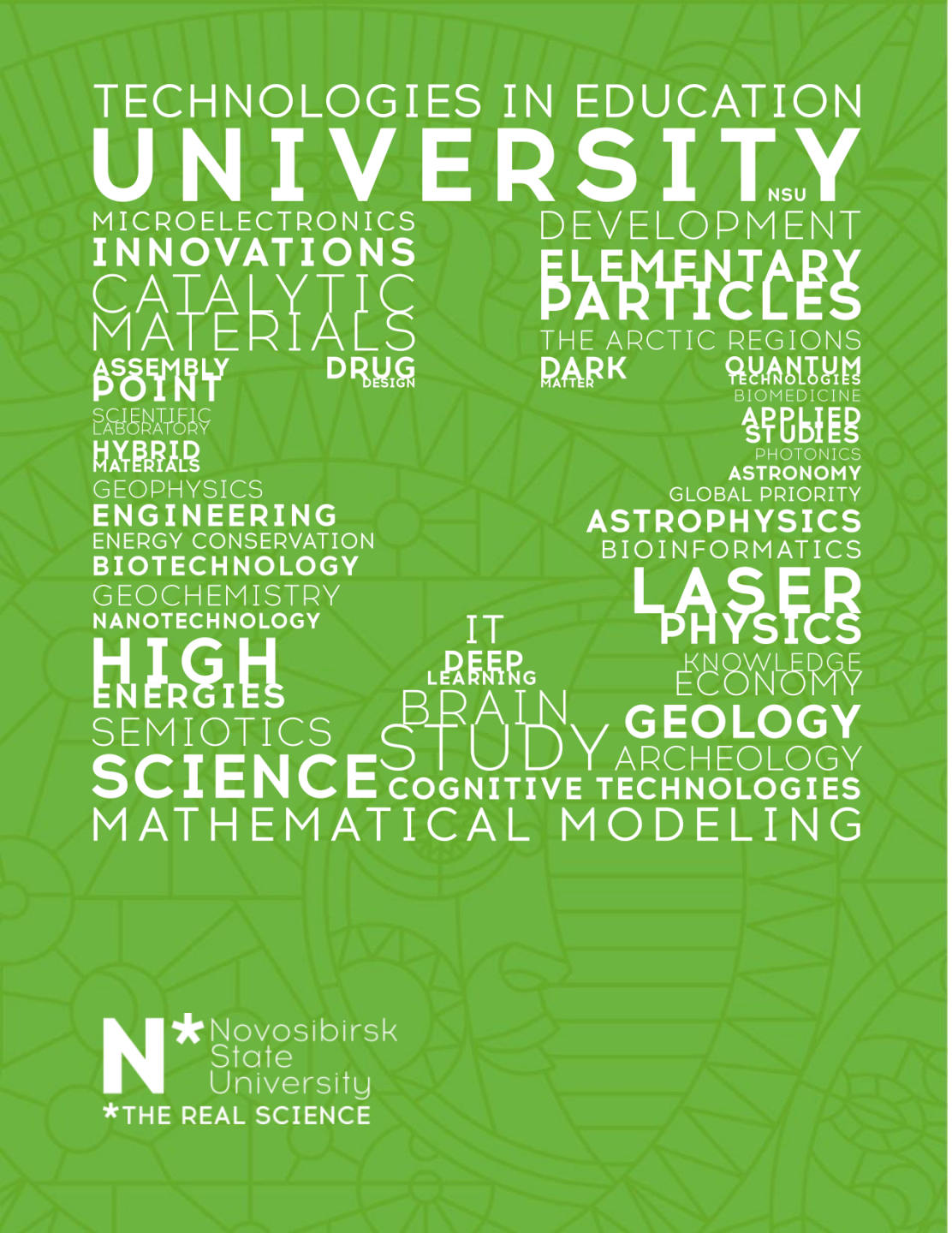


Базовые методы ИИ
Семинар 7

Глушенко Андрей Валерьевич
ФФ НГУ

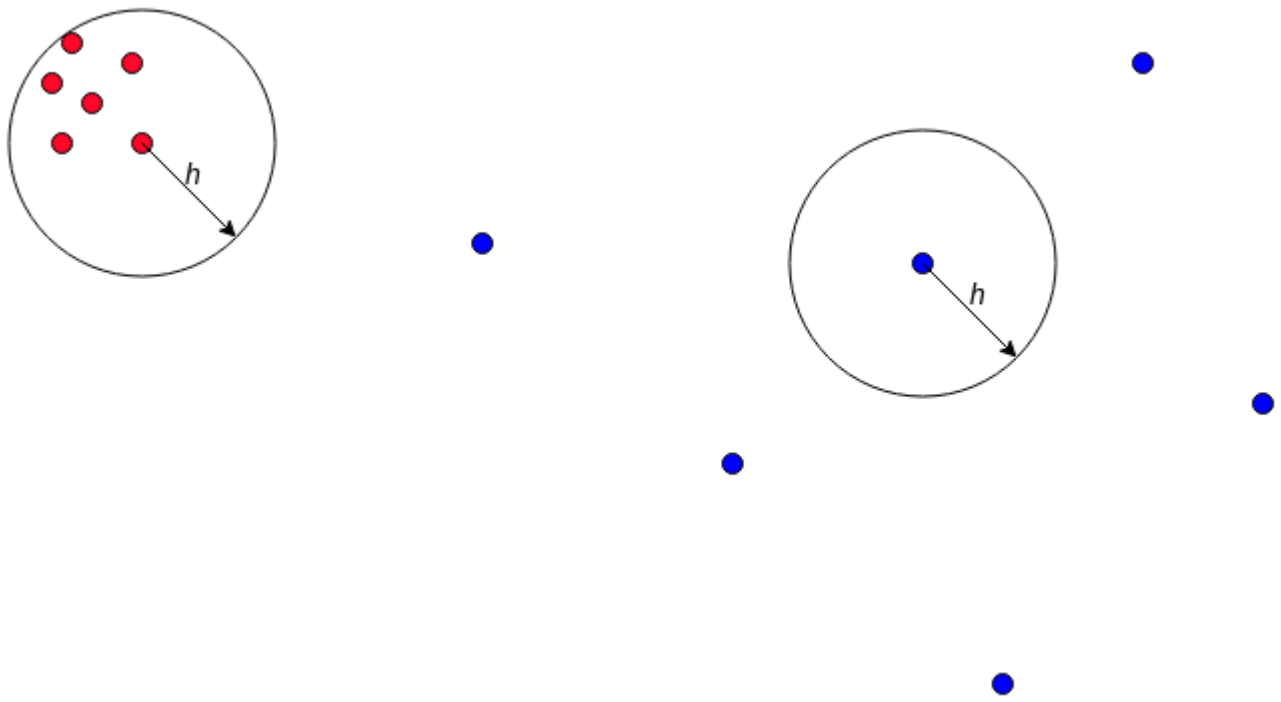
Логическое развитие k-NN

Метод парзеновского окна

Метод потенциальных функций

Метод парзеновского окна (Parzen Window)

Вместо того чтобы фиксировать число соседей k, мы фиксируем ширину окна h — то есть радиус области вокруг новой точки.



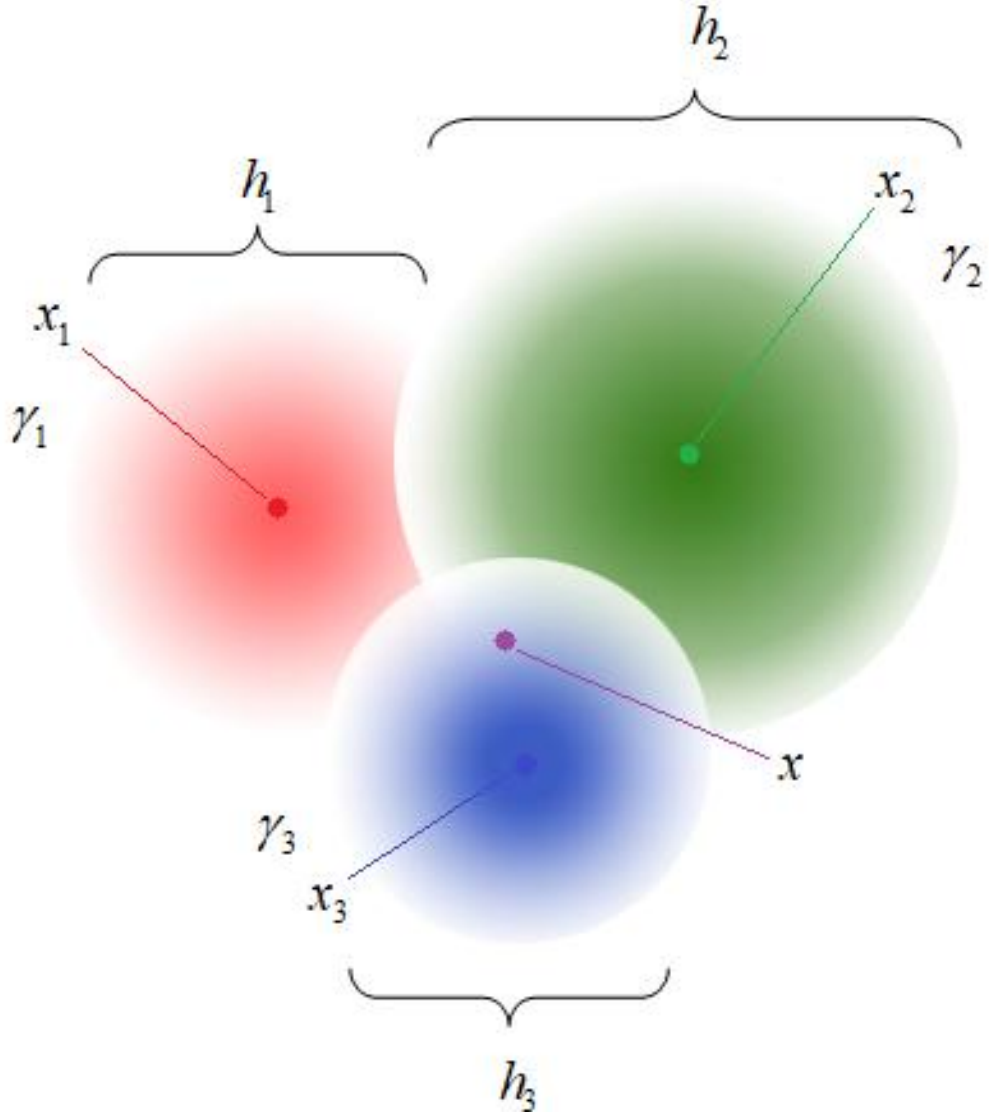
Метод парзеновского окна (Parzen Window)

- **Функция ядра (Kernel):** Чтобы оценка была плавной, используют функцию ядра $K(d)$. Она определяет, какой вес дать соседу в зависимости от расстояния до него.
  - **Прямоугольное ядро:** все, кто внутри радиуса h, имеют вес 1, остальные — 0.
  - **Гауссовское ядро:** вес плавно затухает по мере удаления от центра (даже очень далекие точки теоретически вносят микроскопический вклад).
- **Главная сложность:** Подобраться параметр h. Если окно слишком узкое — классификация будет дерганой и шумной; если слишком широкое — всё превратится в «среднюю температуру по больнице».

Метод потенциальных функций

Этот метод можно представить физически. Представим, что каждая точка обучающей выборки — это заряженная частица, которая создает вокруг себя «энергетическое поле» (потенциал).

Когда мы подносим новую точку в пространство, мы измеряем суммарный потенциал, который на неё действует со стороны классов «А», «Б» и так далее. К какому классу «поле» притягивает сильнее, туда точка и относится.



Отличие метода потенциальных функций от метода парзеновского окна

- У каждой точки обучающей выборки может быть свой «заряд» (вес) γ_i.
- Этот заряд подбирается в процессе обучения. Если точка часто ошибается при классификации других, её «заряд» (важность) увеличивают.

Зачем это нужно: Это позволяет алгоритму выделять наиболее важные, «эталонные» объекты и игнорировать бесполезные или шумовые точки.

Резюмируем

- **k-NN:** «Учитываем k ближайших соседей, неважно как далеко они стоят».
- **Парзеновское окно:** «Учитываем всех, кто стоит в радиусе h. Если там никого нет — оказываемся в тупике».
- **Потенциальные функции:** «Каждая точка излучает влияние. Суммируем, чьё излучение в этой локации мощнее».

Математические функции ядер

В математике метрических методов ядро $K(z)$ — это функция, которая определяет «ценность» соседа в зависимости от расстояния до него. Чем ближе объект (меньше z), тем выше значение функции.

Обычно $z = \frac{\rho(x,x_i)}{h}$, где ρ - расстояние, h - ширина окна (радиус).

Чтобы функция считалась ядром, она должна удовлетворять следующим свойствам:

- Неотрицательность: $K(z) \geq 0$,
- Симметричность: $K(-z) = K(z)$
- Нормированность: $\int K(z)dz = 1$ (часто, но не всегда)
- Максимальная в нуле: $K(0)$ - пик, а при удалении от нуля значение падает.

Виды ядер

- **Прямоугольное (Uniform):**
  - Самое простое: дает вес 1 всем, кто попал внутрь окна, и 0 всем остальным.
  - Работает как обычное усреднение. Граница классификации получается «рваной» и ступенчатой.

$$K(z) = \frac{1}{2}[|z| \leq 1]$$

- **Треугольное (Triangular):**
  - Вес линейно падает до нуля при достижении границы окна.
  - Более гладкое, чем прямоугольное. Хорошо подходит, когда нужно быстро считать, но учитывать близость.

$$K(z) = (1 - z)[|z| \leq 1]$$

Виды ядер

- **Гауссовское (Gaussian):**
  - Самое «гладкое». В отличие от предыдущих, оно бесконечное — то есть даже очень далекие точки имеют ненулевой вес, хотя он и стремится к нулю экспоненциально быстро.
  - Дает идеально плавные границы классов. Математически удобно, так как функция бесконечно дифференцируема.

$$K(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2}$$

Как это влияет на классификацию?

Выбор ядра влияет на гладкость решающей поверхности, но параметр h (ширина окна) гораздо важнее самого типа ядра:

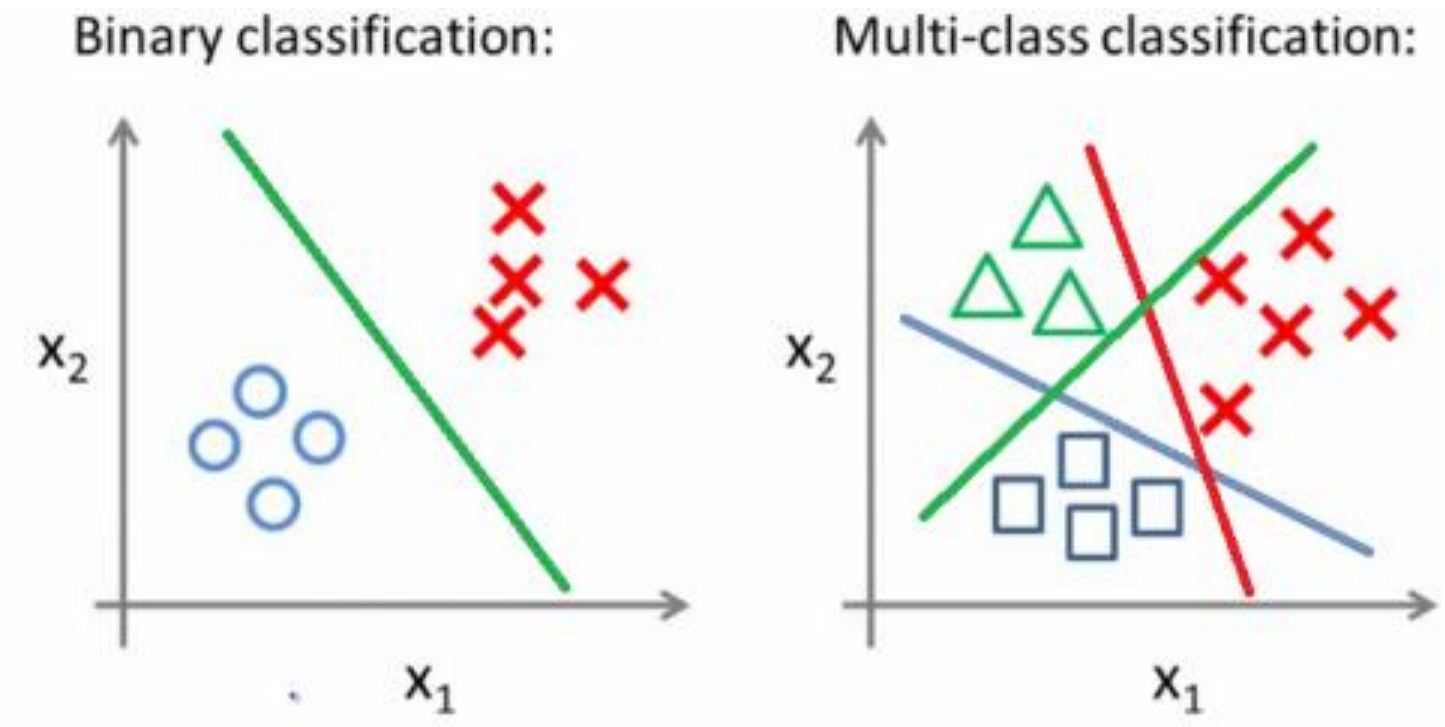
- Если h маленькое — ядро «острое», модель подстраивается под каждый чих данных.
- Если h большое — ядро «размазанное», модель видит только общие тренды.

Многоклассовая классификация

One-vs-Rest (или One-vs-All)

One-vs-One (или All-vs-All)

Когда алгоритм (например, SVM (разберём в будущем) или логистическая регрессия) умеет разделять только **два** класса (бинарная классификация), а в данных их три и более, используют стратегии-надстройки.



One-vs-Rest (One-vs-All, OvA)

Это самый популярный подход. Если у нас есть N классов, мы строим N классификаторов. Иными словами, каждый классификатор обучается отделять «свой» класс от «всех остальных вместе взятых».

Имеем 3 класса: кот, пес, попугай.

- Модель 1: Класс «Кот» против «Не Кот».
- Модель 2: Класс «Пес» против «Не Пес».
- Модель 3: Класс «Попугай» против «Не Попугай».

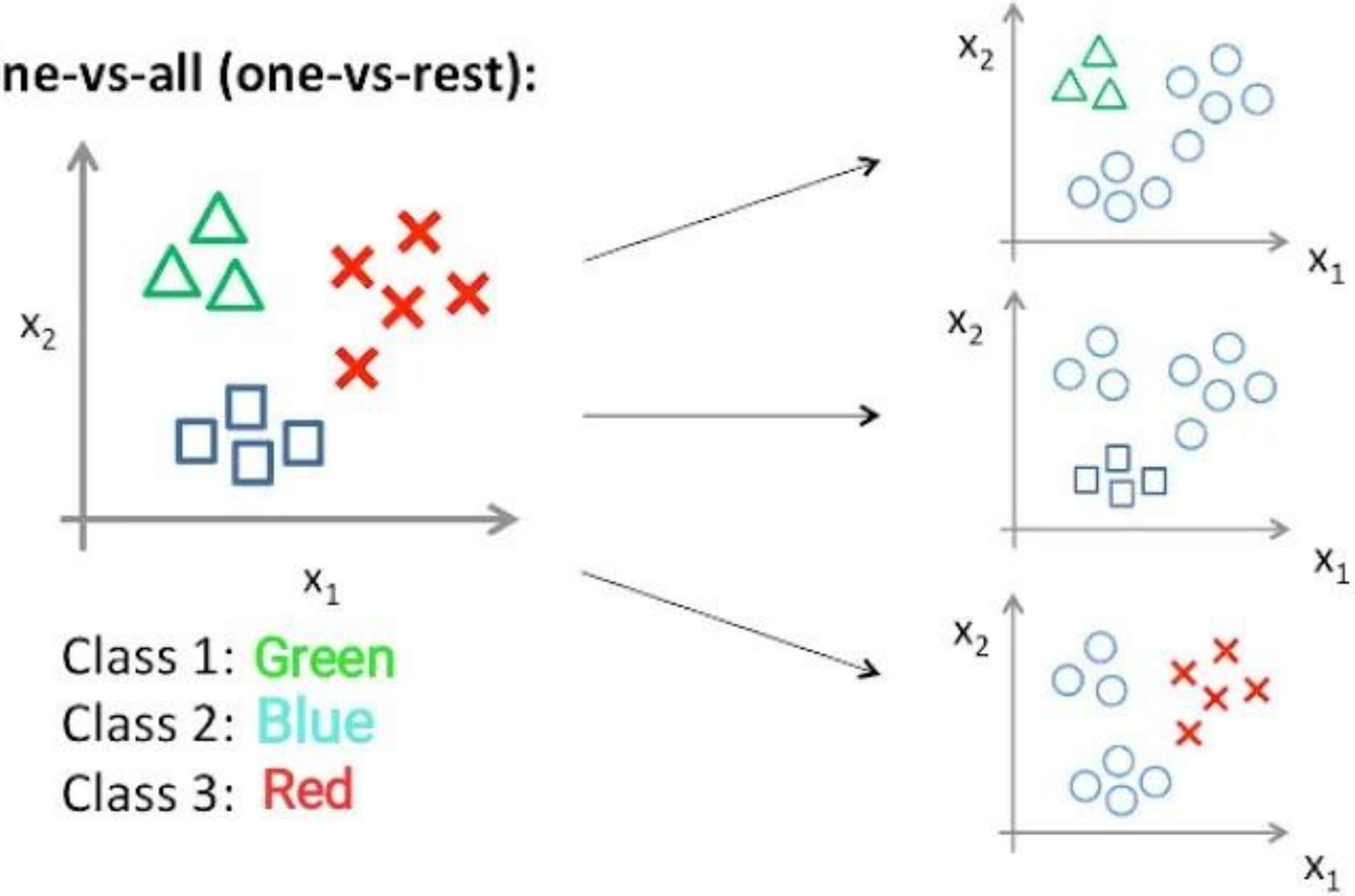
Принятие решения: Мы прогоняем новый объект через все N моделей. Побеждает тот класс, чья модель выдала самую высокую вероятность.

Плюсы: Быстро (всего N моделей).

Минусы: Часто возникает дисбаланс классов (в модели «Кот vs Все» категория «Все» будет намного больше).

Используется при логистической регрессии.

One-vs-all (one-vs-rest):



Class 1: Green
Class 2: Blue
Class 3: Red

One-vs-One (All-vs-All, OvO)

Здесь мы создаем классификатор для каждой пары классов. Как это работает: Если классов 3, моделей будет 3. Если классов 10, моделей будет 45.

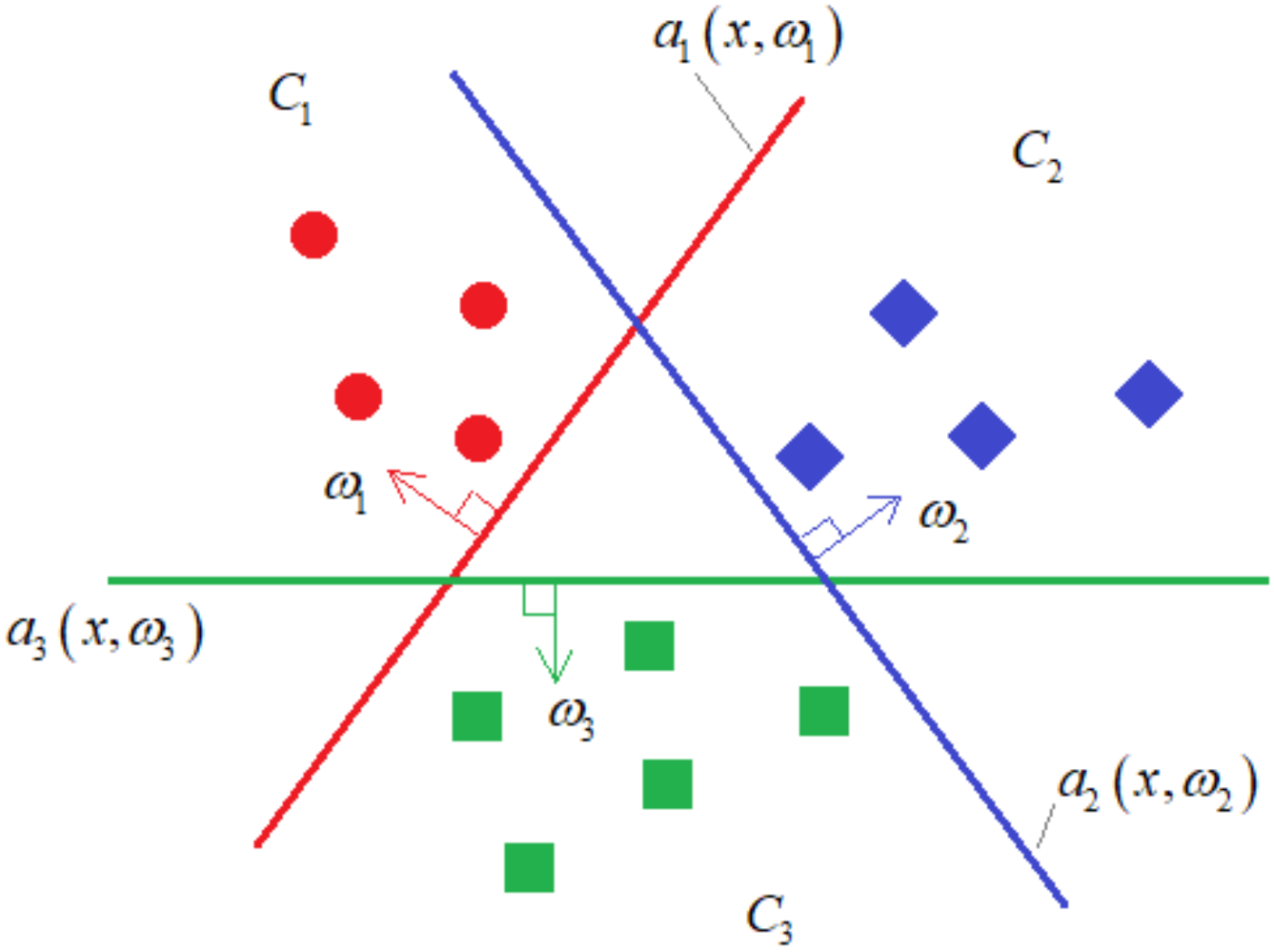
Количество рассчитывается по формуле $\frac{N(N-1)}{2}$.

- Модель 1: Кот vs Пес.
- Модель 2: Кот vs Попугай.
- Модель 3: Пес vs Попугай.

Принятие решения: Объект прогоняется через все пары. Каждый классификатор отдает «голос» за одного из двух участников. Побеждает класс, набравший больше всего голосов (режим «парламентских выборов»).

Плюсы: Нет проблемы дисбаланса, так как в каждой паре обычно сравнимое число примеров.

Минусы: Требует огромных вычислительных ресурсов, если классов очень много (например, 100 классов = 4950 моделей).

Используется в SVM (ядерный метод).

One-vs-One (All-vs-All, OvO)